

Technical Documentation

Embedded Core Model Codebase

A Complete Reference for All Files, Functions, Hyperparameters,
and Mathematical Foundations

Ali Q.

Based on Prebeck (2018), supervised by Dr. Claus Metzner

April 17, 2026

Contents

1	Background: Prebeck (2018) Thesis Summary	2
1.1	Research question	2
1.2	Main finding	2
1.3	Mathematical framework used by Prebeck	2
1.3.1	Boltzmann neuron model	2
1.3.2	Noise model (different input)	2
1.3.3	Mutual information	2
1.3.4	Lognormal weight distribution	3
1.3.5	Temperature parameter t_0	3
2	File Structure	3
3	src/weights/prebeck_125node.py	3
3.1	Purpose	3
3.2	Function: <code>build_prebeck_125node_weight_matrix</code>	3
3.2.1	Signature	3
3.2.2	Parameters	4
3.2.3	Node index convention	4
3.2.4	Weight matrix regions	4
3.2.5	Construction algorithm	4
3.2.6	Returns	5
4	src/core/dynamics_prebeck.py	5
4.1	Purpose	5
4.2	Class: <code>PrebeckBoltzmannDynamics</code>	5
4.3	Method: <code>step(s, W, rng, sigma, noise_mode)</code>	5
4.3.1	Full mathematical description	5
4.3.2	Why unipolar, not bipolar?	6
4.3.3	Why synchronous updates?	6
5	src/core/simulate_states.py	6
5.1	Purpose	6
5.2	Function: <code>run_recorded_states</code>	6
5.2.1	Signature	6

5.2.2	Parameters	7
5.2.3	Why burn-in?	7
5.2.4	Algorithm	7
6	src/metrics/info.py	7
6.1	Function: <code>entropy_base2(p)</code>	7
6.2	Function: <code>mutual_information_joint(p_joint)</code>	7
7	src/metrics/motif_mi.py	8
7.1	Triplets used in experiments	8
7.2	Function: <code>encode_motif_codes</code>	8
7.3	Function: <code>pooled_joint_intra</code>	8
7.4	Function: <code>pooled_joint_inter</code>	9
8	Hyperparameters in <code>simulation.py</code>	9
9	Key Mathematical Results and Relationships	10
9.1	Stochastic resonance mechanism	10
9.2	Why the embedded network exceeds Gaussian noise	10
9.3	Lognormal distribution properties	10
9.4	Temperature and the sigmoid	10
9.5	Feedback loop analysis (Goal 5)	11
9.6	Hard floor in inter-node logistic probabilities	11
9.7	Pearson correlation coefficient	11
9.8	Lag-time autocorrelation	11
10	Critical Implementation Details (Bug Fixes)	11
10.1	Bug 1: Noise ordering	12
10.2	Bug 2: Temperature mask scope	12
10.3	Bug 3: Unipolar vs. bipolar dynamics	12

1 Background: Prebeck (2018) Thesis Summary

1.1 Research question

Karin Prebeck’s Master thesis, “*Noise in Neural Networks*” (FAU Erlangen-Nürnberg, 2018, supervised by Dr. Claus Metzner), investigates how noise affects information transmission in recurrent neural networks (RNNs). The central question is: given that biological neurons are fundamentally stochastic, can noise be *beneficial* rather than detrimental to information processing?

1.2 Main finding

Prebeck found that for networks with a sufficient fraction of excitatory connections, the mutual information (MI) between successive network states does *not* monotonically decrease with increasing noise. Instead, it peaks at an optimal noise level — a phenomenon known as **stochastic resonance** (SR). The key result relevant to our work (her Fig. 3.19) shows that:

- The *isolated* skeleton with external Gaussian noise exhibits SR: MI forms an inverted-U curve with a peak at $\sigma \approx 2\text{--}3$.
- The skeleton *embedded* in the full 125-node network (with sea and inhibitory nodes, no external noise) achieves *higher* MI than the best possible Gaussian noise can provide.

This suggests that the biological network architecture may be optimised to provide tailored internal noise — more effective than unstructured external fluctuations.

1.3 Mathematical framework used by Prebeck

1.3.1 Boltzmann neuron model

Each neuron i is binary: $s_i \in \{0, 1\}$. The total synaptic input is:

$$z_i(t) = b_i(t) + \sum_j s_j(t) \cdot w_{ji} \quad (1)$$

where $s_j \in \{0, 1\}$ is *unipolar* (silent neurons contribute 0, not -1), w_{ji} is the weight from neuron j to neuron i , and $b_i(t)$ is external noise. The firing probability is:

$$p_i(t+1) = \frac{1}{1 + e^{-z_i(t)/T}} \quad (2)$$

and each neuron fires independently as a Bernoulli trial: $s_i(t+1) \sim \text{Bernoulli}(p_i)$.

1.3.2 Noise model (different input)

External noise is Gaussian and *independent* per neuron:

$$b_i(t) \sim \mathcal{N}(0, \sigma^2) \quad \text{independently for each } i \quad (3)$$

1.3.3 Mutual information

Each 3-node motif can be in one of $2^3 = 8$ states, encoded as integers $0, \dots, 7$. The joint probability matrix $p(i, j)$ counts how often the motif transitions from state i at time t to state j at time $t+1$. Mutual information is:

$$\text{MI} = \sum_{i=1}^8 \sum_{j=1}^8 p(i, j) \cdot \log_2 \frac{p(i, j)}{p_X(i) \cdot p_Y(j)} \quad (4)$$

where $p_X(i) = \sum_j p(i, j)$ and $p_Y(j) = \sum_i p(i, j)$ are the marginals. Maximum possible MI for a 3-node motif is $\log_2 8 = 3$ bits.

1.3.4 Lognormal weight distribution

Synaptic weights are drawn from a lognormal distribution, fitted to Song et al. (2005) cortical layer 5 data:

$$\ln(w) \sim \mathcal{N}(\mu, \sigma_w^2), \quad \mu = -0.702, \quad \sigma_w = 0.9355 \quad (5)$$

This gives a heavy-tailed distribution with a few very strong synapses and many weak ones, consistent with cortical anatomy.

1.3.5 Temperature parameter t_0

Prebeck defines the temperature t_0 as the mean weight of the realized skeleton-internal edges (region A):

$$t_0 = \frac{1}{|A|} \sum_{(i,j) \in A} w_{ij} \quad (6)$$

All inputs are divided by t_0 before the sigmoid, normalising them to units of “average skeleton weights”. The inhibitory weight is set to $w_i = -t_0$, ensuring inhibitory strength is matched to excitatory strength.

2 File Structure

```
src/
  weights/
    prebeck_125node.py    - builds the 125-node weight matrix W
  core/
    dynamics_prebeck.py   - one-step Boltzmann neuron update
    simulate_states.py    - runs simulation and records states
  metrics/
    info.py               - entropy and mutual information
    motif_mi.py           - motif encoding and MI computation
  simulation.py           - master orchestration script
  visualize.py            - figure generation from saved results
```

3 src/weights/prebeck_125node.py

3.1 Purpose

Constructs the full 125×125 weight matrix W for the Embedded Core Model, implementing the architecture of Prebeck Section 2.7.

3.2 Function: build_prebeck_125node_weight_matrix

3.2.1 Signature

```
1 def build_prebeck_125node_weight_matrix(
2     rng, ns, ds,
3     overall_density_ABC=0.116,
4     ne=100, nt=125,
5     mu_logn=-0.702, sigma_logn=0.9355,
6     inhibitory_density_D=0.116,
7     wi_mode="-t0"
8 )
```

3.2.2 Parameters

Parameter	Default	Meaning
rng	—	NumPy Generator . Fixed seed (42) makes the network reproducible.
ns	—	Skeleton size. We use $n_s = 10$ throughout.
ds	—	Skeleton density. Fraction of $n_s(n_s - 1)$ edges realized in region A. At $d_s = 0.35$: $n_A = \lfloor 0.35 \times 90 \rfloor = 31$ edges.
overall_density_ABC	0.116	Total excitatory edge density. With $n_e = 100$ excitatory nodes and $n_i = 25$ inhibitory: total possible = $n_e(n_e - 1) + n_e n_i = 12400$, realized = $\lfloor 0.116 \times 12400 \rfloor = 1438$. Fitted from Song et al. (2005).
ne	100	Excitatory nodes: $n_e = 100 = 10 + 90$.
nt	125	Total nodes: $n_t = 125 = 100 + 25$.
mu_logn	-0.702	Log-space mean of lognormal weight distribution. Fitted from Song et al. (2005).
sigma_logn	0.9355	Log-space standard deviation of lognormal distribution.
inhibitory_density_D	0.116	Density of region D (inhibitory→excitatory). Possible edges: $n_i \times n_e = 2500$. Realized: $\lfloor 0.116 \times 2500 \rfloor = 290$.
wi_mode	"-t0"	Inhibitory weight value: "-t0" sets $w_i = -t_0$ (biological); "-1" sets $w_i = -1$ (control).

3.2.3 Node index convention

$$\underbrace{0, \dots, n_s - 1}_{\text{skeleton}} \quad \underbrace{n_s, \dots, n_e - 1}_{\text{sea}} \quad \underbrace{n_e, \dots, n_t - 1}_{\text{inhibitory}} \quad (7)$$

3.2.4 Weight matrix regions

Region	Connection	Content
A	skeleton → skeleton	Lognormal, strongest weights, density d_s
B	excitatory → sea	Lognormal, remaining weights
C	excitatory → inhibitory	Lognormal, remaining weights
D	inhibitory → excitatory	Uniform $w_i = -t_0$ (or -1), density 11.6%
E	inhibitory → inhibitory	Empty (no connections)

3.2.5 Construction algorithm

Step 1 — Sample excitatory weights. Draw $n_{\text{realized}} = 1438$ weights from the lognormal distribution:

$$w \sim \text{LogNormal}(\mu, \sigma_w^2), \quad \ln(w) \sim \mathcal{N}(-0.702, 0.9355^2) \quad (8)$$

Sort descending: $w_{(1)} \geq w_{(2)} \geq \dots \geq w_{(1438)}$.

Step 2 — Assign top n_A weights to region A. The $n_A = \lfloor d_s \cdot n_s(n_s - 1) \rfloor$ largest weights are randomly distributed among skeleton→skeleton edges. This guarantees the skeleton has the strongest synapses in the network.

Step 3 — Assign remaining weights to regions B and C. The remaining $1438 - n_A$ weights are randomly distributed among all other excitatory edges.

Step 4 — Compute temperature t_0 .

$$t_0 = \frac{1}{n_A} \sum_{k=1}^{n_A} w_{(k)} \quad (9)$$

t_0 is data-dependent: for $d_s = 35\%$, $t_0 \approx 4.59$; for $d_s = 45\%$, $t_0 \approx 4.22$.

Why t_0 equals the mean skeleton weight: dividing all inputs by t_0 normalizes the typical skeleton input to order 1, placing neurons near the sensitive region of the sigmoid function where small changes in input produce meaningful changes in firing probability.

Step 5 — Assign inhibitory weights (region D). $n_D = 290$ randomly chosen inhibitory→excitatory edges receive weight $w_i = -t_0$.

Step 6 — Enforce no self-connections: $W_{ii} = 0$ for all i .

3.2.6 Returns

- **W:** (125×125) float array. $W[j, i]$ is the weight from node j to node i .
- **idx:** dictionary with keys 'skeleton', 'sea', 'inhibitory'.
- **t0:** float, temperature parameter.

4 src/core/dynamics_prebeck.py

4.1 Purpose

Implements one discrete synchronous timestep of the Boltzmann neuron model (Prebeck Eqs. 2.1–2.3).

4.2 Class: PrebeckBoltzmannDynamics

A frozen dataclass with fields:

- **t0:** float — temperature parameter
- **mask:** boolean array of length N — which nodes get divided by t_0

4.3 Method: step(s, W, rng, sigma, noise_mode)

4.3.1 Full mathematical description

Given state $\mathbf{s}(t) \in \{0, 1\}^N$:

Step 1 — Unipolar weighted input:

$$z_i = \sum_{j=1}^N s_j(t) \cdot W_{ji} \quad (10)$$

In matrix form: $\mathbf{z} = \mathbf{s}^\top W$ (shape: $(N,)$). Note: $s_j \in \{0, 1\}$ — inactive neurons contribute exactly *zero*.

Step 2 — Temperature normalization:

$$z_i \leftarrow \frac{z_i}{t_0} \quad \text{for all } i \text{ where } \text{mask}[i] = \text{True} \quad (11)$$

In our implementation, `mask = ones(125)`, so all 125 nodes are normalized.

Critical ordering — normalization before noise: If noise were added before dividing by t_0 , the neuron would receive effective noise $\sigma/t_0 \approx \sigma/4.6$. This would shift the SR peak from $\sigma \approx 2-3$ to $\sigma \approx 12-14$, outside our plotted range.

Step 3 — Add Gaussian noise (different_input mode):

$$z_i \leftarrow z_i + b_i, \quad b_i \sim \mathcal{N}(0, \sigma^2) \quad \text{i.i.d. for each } i \quad (12)$$

Step 4 — Stochastic firing:

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}} \quad (13)$$

$$s_i(t+1) \sim \text{Bernoulli}(p_i) \quad \text{independently for each } i \quad (14)$$

4.3.2 Why unipolar, not bipolar?

The bipolar convention $s_j \in \{-1, +1\}$ would mean a silent neuron actively inhibits its targets. This is biologically incorrect for excitatory neurons, which simply send no signal when silent. The unipolar convention $s_j \in \{0, 1\}$ correctly models this: a silent neuron contributes zero input.

Practically: at $\sigma = 0$ with no external input, the unipolar skeleton locks into a high-firing attractor ($p_{\text{on}} \approx 0.89$). Noise then desaturates this attractor — the mechanism behind SR. Bipolar dynamics would give $p_{\text{on}} \approx 0.5$ at $\sigma = 0$ and produce monotonically decreasing MI, with no SR.

4.3.3 Why synchronous updates?

All N neurons update simultaneously at each timestep, matching Prebeck’s implementation. Asynchronous updates (one neuron at a time) would give different dynamics. Synchronous updates are also computationally efficient since the entire state vector is updated in one matrix operation.

5 src/core/simulate_states.py

5.1 Purpose

Runs the full simulation and records the binary state of a specified subset of neurons over T timesteps.

5.2 Function: run_recorded_states

5.2.1 Signature

```
1 def run_recorded_states(
2     W, dynamics, T, burn_in, sigma, noise_mode,
3     seed, record_idx, init="random"
4 )
```

5.2.2 Parameters

Parameter	Meaning
<code>W</code>	$(N \times N)$ weight matrix
<code>dynamics</code>	<code>PrebeckBoltzmannDynamics</code> instance
<code>T</code>	Number of timesteps to record after burn-in
<code>burn_in</code>	Initial timesteps discarded to reach stationarity
<code>sigma</code>	Gaussian noise standard deviation
<code>noise_mode</code>	'different_input': independent per-neuron noise
<code>seed</code>	Integer random seed for full reproducibility
<code>record_idx</code>	Indices of nodes to record (e.g. skeleton only)

5.2.3 Why burn-in?

The network is initialized randomly. For the first $\sim 10,000$ timesteps, the dynamics are transient — the Markov chain has not yet reached its stationary distribution. Recording during this period biases all statistics. The burn-in discards these transient steps.

Choosing `burn_in`: It must be long enough for the network to “forget” its initial state. The mixing time depends on the spectral gap of the transition matrix. For our network, $T_{\text{burn}} = 10,000$ is sufficient. Increasing it wastes computation; decreasing it too much biases MI estimates.

5.2.4 Algorithm

1. Initialize: $\mathbf{s}(0) \sim \text{Uniform}(\{0, 1\}^N)$
2. Discard T_{burn} steps without recording
3. For $t = 1, \dots, T$: update $\mathbf{s}(t)$, record $\mathbf{s}(t)[\text{record_idx}]$
4. Return $(T \times K)$ array, $K = |\text{record_idx}|$

6 src/metrics/info.py

6.1 Function: `entropy_base2(p)`

Shannon entropy in bits:

$$H = - \sum_{i:p_i>0} p_i \log_2 p_i \quad (15)$$

Input $\mathbf{p}$ is automatically normalized to sum to 1, so raw counts can be passed directly.

For K equally probable states: $H_{\text{max}} = \log_2 K$. For a 3-node motif: $H_{\text{max}} = \log_2 8 = 3$ bits.

6.2 Function: `mutual_information_joint(p_joint)`

Mutual information in bits from an 8×8 joint probability matrix:

$$\text{MI}(X; Y) = \sum_{i=1}^8 \sum_{j=1}^8 p(i, j) \log_2 \frac{p(i, j)}{p_X(i) p_Y(j)} \quad (16)$$

where:

$$p_X(i) = \sum_{j=1}^8 p(i, j) \quad (\text{marginal at time } t) \quad (17)$$

$$p_Y(j) = \sum_{i=1}^8 p(i, j) \quad (\text{marginal at time } t + 1) \quad (18)$$

Equivalent formulation via entropy:

$$\text{MI}(X; Y) = H(X) + H(Y) - H(X, Y) \quad (19)$$

Properties: $\text{MI} \geq 0$, equals 0 iff $X \perp Y$, equals $H(X)$ iff Y is a deterministic function of X .

Numerical safeguards: the ratio $p(i, j)/(p_X(i)p_Y(j))$ is computed only where both numerator and denominator are positive. Small negative values from floating-point errors are clipped to 0.

7 src/metrics/motif_mi.py

7.1 Triplets used in experiments

$$\text{TRIPLETS} = [[0, 1, 2], [3, 4, 5], [6, 7, 8]] \quad (20)$$

Three fixed disjoint triplets from the 10 skeleton nodes (node 9 unused). Each triplet has $2^3 = 8$ possible states.

Why fixed disjoint triplets? Overlapping triplets create statistical dependencies between MI estimates. Disjoint triplets give independent measurements. Using all $\binom{10}{3} = 120$ triplets would be expensive and most share nodes. Three disjoint triplets give a clean, interpretable result matching Prebeck’s analysis.

7.2 Function: encode_motif_codes

Converts binary neuron states to integer codes $0, \dots, 7$:

$$\text{code}(s_a, s_b, s_c) = s_a \cdot 1 + s_b \cdot 2 + s_c \cdot 4 \quad (21)$$

This is binary encoding: $(0, 0, 0) \rightarrow 0$, $(1, 0, 0) \rightarrow 1$, $(0, 1, 0) \rightarrow 2$, $\dots$, $(1, 1, 1) \rightarrow 7$.

Returns array of shape (M, T) where M = number of triplets.

7.3 Function: pooled_joint_intra

Builds an 8×8 joint count matrix:

$$C[i, j] = \sum_{t=1}^{T-1} \sum_{k=1}^M \mathbf{1}[\text{code}_k(t) = i \wedge \text{code}_k(t+1) = j] \quad (22)$$

pooled over all M triplets. The resulting matrix is normalized and passed to `mutual_information_joint`.

Intra-MI measures: How predictable is a triplet’s next state from its current state? High intra-MI means the triplet cycles through states in a structured, repeating pattern (temporal autocorrelation).

7.4 Function: pooled_joint_inter

Builds an 8×8 joint count matrix for directed cross-triplet transitions:

$$C[i, j] = \sum_{t=1}^{T-1} \sum_{(A, B) \in \text{pairs}} \mathbf{1}[\text{code}_A(t) = i \wedge \text{code}_B(t+1) = j] \quad (23)$$

With 3 triplets, there are $3 \times 2 = 6$ ordered pairs: (A, B) , (A, C) , (B, A) , (B, C) , (C, A) , (C, B) .

Why 6 ordered pairs, not 3? Because $I(A(t); B(t+1)) \neq I(B(t); A(t+1))$ in general. The first measures directed temporal influence from A to B ; the second from B to A . Since the weight matrix is asymmetric, these genuinely differ. Averaging all 6 gives a symmetric measure of inter-triplet coordination.

Inter-MI measures: How much does triplet A 's state at time t predict triplet B 's state at time $t+1$? High inter-MI means information flows between sub-groups (functional connectivity).

8 Hyperparameters in simulation.py

Parameter	Value	Type	Justification
NS	10	Architecture	Skeleton size from Prebeck. Small enough to analyze, large enough for 3 disjoint triplets.
DS	0.35	Architecture	Skeleton density. Gives $t_0 \approx 4.59$, SR peak clearly visible. Paper uses $ds=35\%$ throughout.
SEED	42	Reproducibility	Fixed seed. All results are fully reproducible from a single number.
BURN_IN	10,000	Simulation	Sufficient for network to reach stationarity. Increasing it wastes compute without changing results.
T_FULL	1,000,000	Simulation	Main simulations. Gives stable MI estimates and smooth histograms.
T_SWEEP	400,000	Simulation	Per-step for sweeps. Reduced to keep runtime manageable across many values. Still gives stable MI.
SIGMA_VALS	0 to 10, step 0.5	SR sweep	Covers SR peak at $\sigma \approx 2-3$ with sufficient resolution.
V_VALS	-5 to +5, step 0.25	Bias sweep	MI peak at $v \approx -1.75$ for $ds=35\%$; range $[-5, 5]$ captures the full curve.
WI_FRACS	-2 to 0, 11 steps	w_i sweep	Range $[-2t_0, 0]$ brackets the optimum near $-t_0$.
DT_MIN, DT_MAX	-10, +10	Autocorr	± 10 timesteps captures whether correlations decay within one lag or persist.
TRIPLETS	$[[0, 1, 2], [3, 4, 5], [6, 7, 8]]$	Analysis	Fixed disjoint triplets. Node 9 unused. See Section 7.

9 Key Mathematical Results and Relationships

9.1 Stochastic resonance mechanism

1. At $\sigma = 0$: strong recurrent excitation traps the skeleton in a high-firing attractor. $p_{\text{on}} \approx 0.89$, entropy is low, MI is low.
2. At optimal $\sigma \approx 2$ –3: noise desaturates the attractor. The skeleton explores all $2^{10} = 1024$ states more uniformly. Entropy and MI are maximized.
3. At large σ : noise overwhelms synaptic structure. Neurons fire randomly. $\text{MI} \rightarrow 0$.

The result is an inverted-U shaped MI vs. σ curve — the SR signature.

9.2 Why the embedded network exceeds Gaussian noise

The inter-nodes produce *structured* inhibitory fluctuations that are:

- **Temporally correlated**: sea nodes have recurrent excitatory connections, so their activity has memory. This propagates through 252 sea→inter edges.
- **Asymmetric**: always inhibitory ($w_i = -t_0 < 0$), always pushing the skeleton away from its saturated high-firing state.
- **Matched to the skeleton’s timescale**: same Boltzmann dynamics and same weight scale.

In contrast, Gaussian noise is: temporally uncorrelated, symmetric (equally likely up or down), and independent across neurons. The structured inter-node input is analogous to *colored noise* optimally matched to the system, which information theory predicts should outperform white noise.

9.3 Lognormal distribution properties

If $\ln(w) \sim \mathcal{N}(\mu, \sigma_w^2)$, then:

$$\mathbb{E}[w] = e^{\mu + \sigma_w^2/2} = e^{-0.702 + 0.438} \approx 0.740 \quad (24)$$

$$\text{Median}(w) = e^\mu = e^{-0.702} \approx 0.496 \quad (25)$$

$$\text{Var}(w) = (e^{\sigma_w^2} - 1) \cdot e^{2\mu + \sigma_w^2} \quad (26)$$

The heavy right tail means a few very strong synapses and many weak ones — exactly the structure exploited by assigning the top n_A weights to the skeleton.

9.4 Temperature and the sigmoid

The Boltzmann sigmoid with temperature T :

$$p_{\text{fire}} = \frac{1}{1 + e^{-z/T}} \quad (27)$$

- $T \rightarrow 0$: deterministic (Heaviside step function)
- $T \rightarrow \infty$: completely random ($p_{\text{fire}} \rightarrow 0.5$)
- $T = t_0$: neurons with input $z \sim t_0$ have $p_{\text{fire}} = \sigma(1) \approx 0.73$, in a sensitive regime

Dividing z by t_0 is equivalent to setting $T = t_0$ in the above.

9.5 Feedback loop analysis (Goal 5)

The skeleton→inter connection exists in the weight matrix (23 edges) but is functionally negligible because:

$$\frac{|\text{sk} \rightarrow \text{inter edges}|}{|\text{sea} \rightarrow \text{inter edges}|} = \frac{23}{252} \approx 8\% \quad (28)$$

The inter-nodes are driven 92% by the sea. Cutting the skeleton's contribution entirely (Goal 5) produces < 1% change in MI, confirming the sea-inter subsystem acts as a one-way structured noise source, not a feedback loop.

9.6 Hard floor in inter-node logistic probabilities

The logistic probability for inter-nodes satisfies:

$$p_i = \sigma(z_i/t_0) \geq \sigma(0) = 0.5 \quad (29)$$

because inter-nodes only receive *positive* excitatory input (regions B, C). The minimum possible input is $z_i = 0$ (when all presynaptic neurons are silent), giving $\sigma(0) = 0.5$. This explains why the inter-node logistic PDF is bounded strictly away from 0, with no probability mass below $p = 0.5$.

9.7 Pearson correlation coefficient

For two time series $a(t)$ and $b(t)$, the Pearson correlation is:

$$r_{ab} = \frac{\sum_t (a_t - \bar{a})(b_t - \bar{b})}{\sqrt{\sum_t (a_t - \bar{a})^2} \cdot \sqrt{\sum_t (b_t - \bar{b})^2}} \quad (30)$$

The `Pearson_Matrix` function computes this efficiently for all pairs simultaneously using matrix operations, handling the degenerate case of constant signals (where the denominator would be 0) by substituting 1.

9.8 Lag-time autocorrelation

The autocorrelation function at lag Δt for neuron i :

$$C_{xx,i}(\Delta t) = r(X_i(t), X_i(t + \Delta t)) \quad (31)$$

where $r(\cdot, \cdot)$ is the Pearson correlation. Exponentially decaying autocorrelation $C \propto e^{-|\Delta t|/\tau}$ appears as a straight line on a semi-log plot, with τ the correlation time. White noise gives $C(\Delta t) = 0$ for all $\Delta t \neq 0$.

10 Critical Implementation Details (Bug Fixes)

Three bugs were found and fixed during development. Understanding these is essential for defending the implementation.

10.1 Bug 1: Noise ordering

Wrong: add noise before dividing by t_0 :

$$z \leftarrow \frac{\mathbf{s}^\top \mathbf{W} + b}{t_0} = \frac{\mathbf{s}^\top \mathbf{W}}{t_0} + \frac{b}{t_0} \quad (32)$$

Effective noise seen by the neuron: $\sigma_{\text{eff}} = \sigma/t_0 \approx \sigma/4.6$. SR peak at $\sigma \approx 14$.

Correct: divide by t_0 first, then add noise (our implementation):

$$z \leftarrow \frac{\mathbf{s}^\top \mathbf{W}}{t_0} + b \quad (33)$$

Effective noise: $\sigma_{\text{eff}} = \sigma$. SR peak at $\sigma \approx 2\text{--}3$. Matches Prebeck Fig. 3.19.

10.2 Bug 2: Temperature mask scope

Wrong: apply t_0 normalization only to skeleton nodes (10 nodes). Sea and inhibitory nodes receive unnormalized lognormal inputs with $z \approx 1.5+$, causing inter-nodes to over-fire ($p_{\text{on}} \approx 0.69$ instead of 0.62) and over-suppress the skeleton.

Correct: apply t_0 normalization to all 125 nodes (`mask = ones(125)`). This places all populations in the correct dynamic regime, matching Prebeck's reported firing rates: skeleton ≈ 0.52 , sea ≈ 0.29 , inter ≈ 0.62 .

10.3 Bug 3: Unipolar vs. bipolar dynamics

Wrong (bipolar): $s_j \in \{-1, +1\}$ via the mapping $s_j \rightarrow 2s_j - 1$. Gives $p_{\text{on}} \approx 0.5$ at $\sigma = 0$ with monotonically decreasing MI — no SR.

Correct (unipolar): $s_j \in \{0, 1\}$. Gives $p_{\text{on}} \approx 0.89$ at $\sigma = 0$ (saturated attractor), then noise desaturates the system — producing SR. Matches Prebeck Eq. 2.1.